Apellidos: Gómez González
Nombre: Isaac
Asignatura: PCD
Grupo: 2 **Subgrupo:** 3
Convocatoria:
Año académico: 2025–2026
Profesor: Adrián Cánovas Rodríguez

MEMORIA DE PRÁCTICAS

Entregables 2025–2026

Índice

1. Ejercicio 1: ReentrantLock **2**
1.1. Recursos no compartibles 2
1.2. Condiciones de sincronización 2
1.3. Pseudocódigo comentado 2
1.4. Cuestiones planteadas en los ejercicios 4
1.5. Código Java de los ejercicios comentado 5

2. Ejercicio 2: Semáforos **8**
2.1. Recursos no compartibles 8
2.2. Condiciones de sincronización 8
2.3. Pseudocódigo comentado 9
2.4. Cuestiones planteadas en los ejercicios 11
2.5. Código Java de los ejercicios comentado 12

3. Ejercicio 3: Monitores **15**
3.1. Recursos no compartibles 15
3.2. Condiciones de sincronización 15
3.3. Pseudocódigo comentado 16
3.4. Cuestiones planteadas en los ejercicios 22
3.5. Código Java de los ejercicios comentado 23

4. Ejercicio 4: Paso de Mensajes Asíncrono **24**
4.1. Recursos no compartibles 24
4.2. Condiciones de sincronización 24
4.3. Pseudocódigo comentado 25
4.4. Cuestiones planteadas en los ejercicios 28
4.5. Código Java de los ejercicios comentado 29

1. Ejercicio 1: ReentrantLock

1.1. Recursos no compartibles

Unicamente la pantalla, ya que es el recurso que ambos hilos necesitan para imprimir sus resultados.

1.2. Condiciones de sincronización

El uso de un cerrojo para bloquear la sección crítica de la pantalla. Que permite a los hilos el acceso a la pantalla en exclusión mutua, evitando así que ambos hilos impriman al mismo tiempo y mezclen sus salidas.

1.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
// Variables globales
Datos:
cerrojoPantalla : Cerrojo
NUM_OPERACIONES : Entero
LONGITUD_FILAS_Y_COLUMNAS : Entero
// Inicialización de variables globales
NUM_OPERACIONES ← 10
LONGITUD_FILAS_Y_COLUMNAS ← 3
Función main()
    initial (cerrojoPantalla, 1)
    cobegin
        suma()
        multiplicar()
    coend
fin
```

```
Algoritmo: Clase SumaMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Entero[][]
impactoAcumulado : Entero[][]
Función suma()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular suma de matrices
        // Guardar resultado en impactoAcumulado

        wait(cerrojoPantalla)
        // Impresión
        signal(cerrojoPantalla)
    fin
fin
```

```
Algoritmo: Clase CuadradoMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Entero[][]
impactoAcumulado : Entero[][]
Función multiplicar()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular cuadrado de matrices
        // Guardar resultado en impactoAcumulado

        wait(cerrojoPantalla)
        // Impresión
        signal(cerrojoPantalla)
    fin
fin
```

1.4. Cuestiones planteadas en los ejercicios

a) **Si no usas sincronización, ¿cómo podría ser la salida por pantalla?**

Como regla general, si no usas sincronización, la salida por pantalla es caótica e impredecible. En este caso, dado que ambos hilos (SumaMatriz y CuadradoMatriz) están accediendo a la pantalla para imprimir sus resultados, sin sincronización podrían intercalarse las impresiones de ambos hilos. Esto podría resultar en una salida mezclada donde partes de la matriz de SumaMatriz se imprimen junto con partes de la matriz de CuadradoMatriz, haciendo que sea difícil o imposible entender los resultados.

En el caso particular de java cada System.out.println() es una operación atómica, lo que significa que cada llamada a esta función se ejecutará completamente sin ser interrumpida por otro hilo. Sin embargo, esto no garantiza que las impresiones de ambos hilos no se mezclen, ya que cada hilo podría imprimir varias líneas de la matriz antes de que el otro hilo tenga la oportunidad de imprimir. Por lo tanto, aunque cada línea individual se imprima de manera atómica, la secuencia completa de impresiones podría resultar en una salida caótica si no se sincroniza adecuadamente.

1.5. Código Java de los ejercicios comentado

Clase Main

```java
package ejercicio1;

import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;

public class Main {
    public static final int NUM_OPERACIONES = 10;
    public static final int LONGITUD_FILAS_Y_COLUMNAS = 3;
    public static final int NUM_0_9 = 10;

    /* Este cerrojo se encarga de bloquear la sección crítica de la pantalla para que no acceda
    * más de un hilo a la vez */
    public static ReentrantLock cerrojoPantalla = new ReentrantLock();

    /**
    * Función que le da valores aleatorios entre 0 y 9 a una matriz de 3x3,
    * @param matriz Es la matriz que se rellenara con núevos valores aleatorios
    */
    public static void generarMatriz(int[][] matriz){
        for(int i = 0; i < LONGITUD_FILAS_Y_COLUMNAS; i++) {
            for(int j = 0; j < LONGITUD_FILAS_Y_COLUMNAS; j++) {
                Random random = new Random();
                matriz[i][j] = random.nextInt(NUM_0_9);
            }
        }
    }


    /**
    * Función principal del programa donde se ejecutan dos hilos, uno que se encarga de
    * hacer el cuadrado de una matriz y otro el doble de una matriz.
    * @param args
    */
    public static void main(String[] args) {
        CuadradoMatriz cuadrado = new CuadradoMatriz();
        SumaMatriz suma = new SumaMatriz();
        cuadrado.start();
        suma.start();

        try {
            cuadrado.join();
            suma.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Clase SumaMatriz

```
package ejercicio1;

public class SumaMatriz extends Thread{


    @Override
    public void run() {

        int metricasDeEntrenamiento[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];
        int impactoAcumulado[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];

        for (int operacionesRealizadas = 0;
            operacionesRealizadas < Main.NUM_OPERACIONES; operacionesRealizadas++) {

            Main.generarMatriz(metricasDeEntrenamiento);

            // Suma de la matriz A + A (2A)
            for (int i = 0; i < Main.LONGITUD_FILAS_Y_COLUMNAS; i++) {
                for (int j = 0; j < Main.LONGITUD_FILAS_Y_COLUMNAS; j++) {
                    impactoAcumulado[i][j] =
                        metricasDeEntrenamiento[i][j] + metricasDeEntrenamiento[i][j];
                }
            }


            /* Aquí se bloquea el cerrojo de la pantalla para asegurar que este recurso no
            * compartible no sea usado por más de un proceso a la vez
            */
            Main.cerrojoPantalla.lock();

            System.out.println("A␣+␣A\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }
            System.out.println("\n+\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }

            System.out.println("\n2A\n");

            for (int[] fila : impactoAcumulado) {
                for (int valor : fila) {
                    System.out.print(valor + "r␣");
                }
                System.out.println();
            }
            System.out.println();

            Main.cerrojoPantalla.unlock();

        }

    }
}
```

Clase CuadradoMatriz

```
package ejercicio1;

public class CuadradoMatriz extends Thread {

    @Override
    public void run() {

        int metricasDeEntrenamiento[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];
        int impactoAcumulado[][] =
            new int[Main.LONGITUD_FILAS_Y_COLUMNAS][Main.LONGITUD_FILAS_Y_COLUMNAS];

        for (int operacionesRealizadas = 0;
            operacionesRealizadas < Main.NUM_OPERACIONES; operacionesRealizadas++) {

            Main.generarMatriz(metricasDeEntrenamiento);

            // Cálculo del cuadrado de la matriz
            for (int i = 0; i < Main.LONGITUD_FILAS_Y_COLUMNAS; i++) {
                for (int j = 0; j < Main.LONGITUD_FILAS_Y_COLUMNAS; j++) {
                    impactoAcumulado[i][j] = 0;
                    for (int k = 0; k < Main.LONGITUD_FILAS_Y_COLUMNAS; k++) {
                        impactoAcumulado[i][j] +=
                            metricasDeEntrenamiento[i][k] * metricasDeEntrenamiento[k][j];
                    }
                }
            }


            /*
            * Aquí se bloquea el cerrojo de la pantalla para asegurar que este recurso no
            * compartible no sea usado por más de un proceso a la vez
            */
            Main.cerrojoPantalla.lock();

            System.out.println("A␣x␣A\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }
            System.out.println("\nx\n");

            for (int[] fila : metricasDeEntrenamiento) {
                for (int valor : fila) {
                    System.out.print(valor + "a␣");
                }
                System.out.println();
            }

            System.out.println("\nA2\n");

            for (int[] fila : impactoAcumulado) {
                for (int valor : fila) {
                    System.out.print(valor + "r␣");
                }
                System.out.println();
            }
            System.out.println();

            Main.cerrojoPantalla.unlock();

        }

    }
}
```

2. Ejercicio 2: Semáforos

2.1. Recursos no compartibles

Para este problema cada panel individual no es compartible, ya que cada uno se comporta como una pantalla independiente, por ello es necesario protegerlos. Para protegerlos se crea una variable **panelesLibres** que almacena un booleano para cada panel, que también es un recurso no copartible, porque podría estar siendo usa tanto por un hilo para leer como por otroç para escribir.

2.2. Condiciones de sincronización

Para garantizar las condiciones de sincronización se crean dos semáforos: **semaforoDePaneles** y **mutexVariablesCompartidas**. El primero se encarga de bloquear la sección crítica de la pantalla para que no accedan más de tres procesos a los paneles al mismo tiempo, siendo necesario que los hilos que acceden a los paneles se bloqueen en la sección crítica de la pantalla si no hay paneles libres y se liberan cuando hay alguno disponible, siendo esto una condición de sincronización. Con el segundo semáforo se comprueba que alguíen no esté usando ya el array **panelesLibres** para evitar que uno pueda escribir y otro leer a la vez, protegiendo así un recurso no compartible, hasta que quede libre.

2.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
Datos:
    paneles : Panel[] // Paneles disponibles
    panelesLibres : Booleano[]
    semaforoDePaneles : Semaforo
    mutexVariablesCompartidas : Semaforo
    NUMERO_DE_PANELES : Entero
    NUMERO_DE_EQUIPOS : Entero
    NUMERO_DE_DEPORTISTAS_POR_EQUIPO : Entero
    NUMERO_DE_SECUENCIAS : Entero
    NUMERO_DE_EJERCICIOS : Entero
    equipos : Equipo[]
// Inicialización de variables globales
NUMERO_DE_PANELES ← 3
NUMERO_DE_EQUIPOS ← 10
NUMERO_DE_DEPORTISTAS_POR_EQUIPO ← 4
NUMERO_DE_SECUENCIAS ← 3
NUMERO_DE_EJERCICIOS ← 5
Función main()
    initial(semaforoDePaneles, NUMERO_DE_PANELES)
    initial(mutexVariablesCompartidas, 1)
    para i ← 1 a NUMERO_DE_PANELES hacer
        panelesLibres[i] ← true
    fin
    cobegin
        para i ← 1 a NUMERO_DE_EQUIPOS hacer
            equipos[i].equipo(i)
        fin
    coend
fin
```

```
Algoritmo: Clase Equipo
// Variables de la instancia
Datos:
tablaDeMatricesDeEjercicios : Entero[][][]
MatrizResumenR : Entero[][]
idEquipo : Entero

Función equipo(idEquipo)
    para i ← 1 a NUMERO_DE_SECUENCIAS hacer
        // Generar tablaDeMatricesDeEjercicios
        // Rellenar MatrizResumenR

        wait(semaforoDePaneles)
            wait(mutexVariablesCompartidas)
                // Buscar panel disponible
                // Poner a false el panel escogido
                // Guadar idPanel escogido
            signal(mutexVariablesCompartidas)
            // Imprimir en panel escogido
            wait(mutexVariablesCompartidas)
                panelesLibres [idPanel] = true
            signal(mutexVariablesCompartidas)
        signal(semaforoDePaneles)
    fin
fin
```

2.4. Cuestiones planteadas en los ejercicios

Según las condiciones del problema:

a) **(0.5 puntos sobre 10) Si consideramos un panel solamente ¿cómo es el acceso al mismo? Si consideramos los tres paneles ¿cómo es el acceso? Explica ambas situaciones.**
En el caso de tener un unico panel, el acceso al mismo se realiza mediante un mutex, que se encarga de bloquear la sección crítica de la pantalla para que esta no pueda ser usada por más de un proceso a la vez.

En el caso de los tres paneles, hay que garantizar que no accede más de un hilo a cada panel, pero esto permite que varios hilos impriman sus resultados en paralelo ya que es como si hubieran tres pantallas independientes, por lo que solo hay que proteger la selección de la pantalla que es mucho más rápido que esperar a que un hilo use la pantalla.

b) **(0.5 puntos sobre 10) ¿Qué tipo de semáforos has usado para resolver el problema (binarios, generales)?**
Uno binario para proteger la selección del panel y uno general para controlar el número de paneles disponibles.

2.5. Código Java de los ejercicios comentado

Clase Main

```java
package ejercicio2;

import java.util.Random;
import java.util.concurrent.Semaphore;

public class Main {

    public static final int NUMERO_DE_PANELES = 3;
    public static final int NUMERO_DE_EQUIPOS = 10;
    public static final int NUMERO_DE_DEPORTISTAS_POR_EQUIPO = 4;
    public static final int NUMERO_DE_SECUENCIAS = 3;
    public static final int NUMERO_DE_EJERCICIOS = 5;
    public static final int INICIALIZAR_SEMAFORO_BINARIO = 1;
    public static final int NUM_0_9 = 10;

    public static Panel[] paneles = new Panel[NUMERO_DE_PANELES];
    public static boolean[] panelesLibres = new boolean[NUMERO_DE_PANELES];
    public static Semaphore mutexVariablesCompartidas =
        new Semaphore(INICIALIZAR_SEMAFORO_BINARIO);
    public static Semaphore semaforoDePaneles = new Semaphore(NUMERO_DE_PANELES);

    public static Equipo[] equipos = new Equipo[NUMERO_DE_EQUIPOS];

    /**
    * Función que le da valores aleatorios entre 0 y 9 a una matriz de 5x5,
    * @param matriz Es la matriz que se rellenara con núevos valores aleatorios
    */
    public static void generarMatriz(int[][] matriz){
        for(int i = 0; i < NUMERO_DE_EJERCICIOS; i++) {
            for(int j = 0; j < NUMERO_DE_EJERCICIOS; j++) {
                Random random = new Random();
                matriz[i][j] = random.nextInt(NUM_0_9);
            }
        }
    }

    public static void main(String[] args) {
        //Bucle para generar los paneles uno al lado del otro sin superponerse e
        //inicializo el array de booleanos
        for (int numPanel = 0; numPanel < NUMERO_DE_PANELES; numPanel++) {
            paneles[numPanel] = new Panel(new String("Panel " + (numPanel + 1)),
                (numPanel * 410), 0);
            panelesLibres[numPanel] = true;
        }

        for(int numEquipo = 0; numEquipo < NUMERO_DE_EQUIPOS; numEquipo++) {
            equipos[numEquipo] = new Equipo(numEquipo);
            equipos[numEquipo].start();
        }

        for(int numEquipo = 0; numEquipo < NUMERO_DE_EQUIPOS; numEquipo++) {
            try {
                equipos[numEquipo].join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

}
```

Clase Equipo

```
package ejercicio2;

import java.util.Arrays;

public class Equipo extends Thread{

        public static final int SPRINT = 0;
        public static final int PRIMER_EJERCICIO_MAX = 1;

        private int idEquipo;

        public Equipo(int idEquipo) {
            this.idEquipo = idEquipo;
            }

        /**
        * Función para extraer las mejores marcas de una matriz de ejercicios
        * @param ejercicios
        * @return
        */
        private int[] mejoresMarcas(int[][] ejercicios) {
            int[] vectorDeMarcas = new int[Main.NUMERO_DE_EJERCICIOS];
            //Guardo sprint con el valor mínimo ya que es una pruba de velocidad
            vectorDeMarcas[SPRINT] = Arrays.stream(ejercicios[SPRINT]).min().getAsInt();
            //El resto de valores los guardo con el máximo salto, lanzamiento, resistencia, y
            //agilidad.
            for(int numEjercicios = PRIMER_EJERCICIO_MAX;
                    numEjercicios < Main.NUMERO_DE_EJERCICIOS; numEjercicios++) {
                vectorDeMarcas[numEjercicios] = Arrays.stream(ejercicios[numEjercicios])
                .max().getAsInt();
            }

            return vectorDeMarcas;
        }

        /**
        * Función que recibe una matriz de 4x5 y devuelve un string con el formato necesario
        * para imprimir
        * @param matriz matriz que va a ser transformada en string (4 filas x 5 columnas)
        * @return matriz transformada en string con el formato que necesito
        */
        private static String matrizToString(int[][] matriz) {
            String strMatriz = "";
            for (int i = 0; i < Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO; i++) {
                for (int j = 0; j < Main.NUMERO_DE_EJERCICIOS; j++) {
                    strMatriz += matriz[i][j] + "␣";
                }
                strMatriz += "\n";
            }
            return strMatriz;
        }


        @Override
        public void run() {

            //Estructura con las 4 matrices
            int[][][] tablaDeMatricesDeEjercicios =
                new int[Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO]
                [Main.NUMERO_DE_EJERCICIOS][Main.NUMERO_DE_EJERCICIOS];

            //Matriz con las mejores marcas en cada deporte de cada deportista
            int[][] matrizResumenR = new int[Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO]
                    [Main.NUMERO_DE_EJERCICIOS];


            for(int secuenciasPorEquipo = 0; secuenciasPorEquipo < Main.NUMERO_DE_SECUENCIAS;
                    secuenciasPorEquipo++) {
                //Primero se genera una matriz por jugador
                for(int numJugadores = 0; numJugadores < Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO;
                        numJugadores++) {
                    Main.generarMatriz(tablaDeMatricesDeEjercicios[numJugadores]);
                }

                //Se seleccionan y guardan las mejores marcas de cada jugador
                //sprint, salto, lanzamiento, resistencia, y agilidad
                for(int numJugadores = 0; numJugadores < Main.NUMERO_DE_DEPORTISTAS_POR_EQUIPO;
                        numJugadores++) {
                    matrizResumenR[numJugadores] =
                        mejoresMarcas(tablaDeMatricesDeEjercicios[numJugadores]);
                }
```

```
                //Bloqueo el acceso a los paneles para que solo puedan estar el mismo número de
                //procesos que paneles hay.
                try {
                    Main.semaforoDePaneles.acquire();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                //Bloqueo el acceso a la varible que puede ser usada por más de un proceso a la
                //vez para lectura y escritura, por un semáforo binario.
                try {
                    Main.mutexVariablesCompartidas.acquire();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                //Llegado este punto hay un panel libre asegurado por lo que uso un while para
                //encontrar uno.
                int idPanel = 0;
                while(Main.panelesLibres[idPanel] != true) {
                    idPanel++;
                }
                Main.panelesLibres[idPanel] = false;
                // Dejo libre el semáforo binario
                Main.mutexVariablesCompartidas.release();

                String mensajeParaPanel = new String(
                        "Usando␣panel␣" + idPanel + "␣el␣hilo␣(equipo)␣" + idPanel + "\n\n"
                        + "Matriz␣R␣(Resumen␣del␣equipo␣" + idEquipo + "":␣mejores␣marcas=\n"
                        + matrizToString(matrizResumenR) + "\n"
                        + "Terminando␣de␣usar␣panel␣" + idPanel + "␣el␣hilo␣(equipo)␣"
                        + idEquipo + "\n");

                Main.paneles[idPanel].escribir_mensaje(mensajeParaPanel);

                try {
                    Main.mutexVariablesCompartidas.acquire();
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                Main.panelesLibres[idPanel] = true;
                Main.mutexVariablesCompartidas.release();
                Main.semaforoDePaneles.release();
            }
        }

}
```

3. Ejercicio 3: Monitores

3.1. Recursos no compartibles

No se podrán compartir, cada uno de los tornos y las colas para acceder estos. Por ello un monitor se encargará de gestionar el acceso a estos, el uso y su liberación.

En el caso de las zonas de entrenamiento, tenemos el mismo problema, por lo que se usará un monitor para gestionar el acceso a las zonas y su liberación. Además de esto también se usará el mismo monitor para gestionar el acceso a la bicicleta premium y su liberación, ya que es un recurso no compartible que usa una cola condition independiente.

Además, para poder acceder a la pantalla, se usará un monitor para evitar que entre más de un proceso a esta.

Uso tres monitores porque el código no se complica demasiado y cada uno de los monitores son independientes entre sí. Estos recursos no compartibles podrían estar gestionados de una infinidad de maneras, siempre que se cumpla la exclusión mutua en las secciones críticas.

3.2. Condiciones de sincronización

Para garantizar las condiciones de sincronización, se crean tres monitores: **MonitorTornos**, **MonitorZonas** y **MonitorPantalla**. El primero tiene un protocolo para el acceso y uso de los tornos y otro para su liberación. Evitando que varios procesos accedan simultáneamente a las variables de los tornos.

El segundo monitor tiene un protocolo para el acceso y uso de las zonas, otro para el uso de la bicicleta premium, otro para dejar libre lacola de la bicicleta premium y un último para la liberación de la zona tras su uso. Evitando que varios procesos accedan a las zonas simultáneamente o que varios lean y escriban a la vez donde no deberían.

Por último, el monitor de la pantalla se encarga de recibir lo que tiene que imprimir y usa esto en exclusión mutua, evitando que más de un proceso pueda acceder a la impresión por pantalla al mismo tiempo.

3.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
Datos:
clientes : Cliente[]
monitorTornos : MonitorTornos
monitorZonas : MonitorZonas
monitorPantalla : MonitorPantalla
TIEMPO_MAX_DE_X_TORNO : Entero
TIEMPO_MAX_DE_Y_ZONA : Entero
// Inicialización de variables globales
NUMERO_DE_CLIENTES ← 50
TIEMPO_MAX_DE_X_TORNO ← 5
TIEMPO_MAX_DE_Y_ZONA ← 5
Función main()
    cobegin
        para i ← 1 a NUMERO_DE_CLIENTES hacer
            // Inicialización de un hilo cliente
            clientes[i].cliente(i);
        fin
    coend
fin
```

```
Algoritmo: Clase Cliente
Datos:
X : Entero
Y : Entero
usarBicicletaPremium : Booleano
// Inicializar variables del cliente
X ← random(1,TIEMPO_MAX_DE_X_TORNO)
Y ← random(1,TIEMPO_MAX_DE_Y_ZONA)
usarBicicletaPremium ← false
Función cliente(idCliente : Entero)
    idTorno ← monitorTornos.seleccionarYUsarTorno(X)
    sleep(X)
    monitorTornos.salirDeTorno(X, idTorno)
    idZonaYTiempos ← monitorZonas.accederAZona(Y)
    si idZonaYTiempos[0] = CARDIO entonces
        usarBicicletaPremium ← random(1,10) ≤ 3
    fin
    // Crear mensaje con parámetros obtenidos previamente
    monitorPantalla.usarPantalla(mensaje)
    si usarBicicletaPremium entonces
        monitorZonas.entrarEnBicicletaPremium()
    fin
    sleep(Y)
    si usarBicicletaPremium entonces
        monitorZonas.salirDeBicicletaPremium()
    fin
    monitorZonas.salirDeZona(Y, idZonaYTiempos[0])
fin
```

```
Algoritmo: Monitor Tornos
Datos:
Privado numeroDeTornos : Entero
Privado tiempoEnTornos : Entero[]
Privado tornosLibres : Booleano[]
Privado colasDeTornos : Condition[]
// Inicializar variables del monitor
numeroDeTornos ← 3
para i ← 1 a numeroDeTornos hacer
    tiempoEnTornos[i] ← 0
    tornosLibres[i] ← true
fin
// Funciones del monitor
Función seleccionarYUsarTorno(idTorno : Entero) : Entero
    // Buscar torno libre
    // Si no hay
    // buscar torno con tiempo mínimo
    // Sumar mi tiempo en torno
    si tornosLibres[idTornoSeleccionado] = False entonces
        delay(colasDeTornos [idTornoSeleccionado])
    fin
    tornosLibres[idTornoSeleccionado] ← False
    devolver idTornoSeleccionado
fin
Función salirDeTorno(X : Entero, idTorno : Entero)
    tornosLibres[idTorno] ← True
    tiempoEnTornos[idTorno] ← tiempoEnTornos[idTorno] − X
    resume(colasDeTornos [idTorno])
fin
```

```
Algoritmo: Monitor ZonasGimnasio
Datos:
Privado numeroDeZonas : Entero
Privado maquinasPorZona : Entero
Privado maquinasPorZonaLibres : Entero[]
Privado colasDeZonas : Condition[]
Privado tiemposEnZonas : Entero[]
Privado bicicletaPremiumLibre : Booleano
Privado colaBicicletaPremium : Condition
// Inicializar variables del monitor
numeroDeZonas ← 4
maquinasPorZona ← 5
bicicletaPremiumLibre ← true
para i ← 1 a numeroDeZonas hacer
    maquinasPorZonaLibres[i] ← maquinasPorZona
    tiemposEnZonas[i] ← 0
fin
bicicletaPremiumLibre ← true
// Funciones del monitor
Función accederAZona(Y : Entero) : Entero[]
    // Buscar zona
    // Guardo todos los tiempos
    tiemposEnZonas [idZona] ← tiemposEnZonas [idZona] + Y
    idZonaYTiempos[0] ← idZona
    si maquinasPorZonaLibres[idZona] ≤ 0 entonces
        delay(colasDeZonas [idZona])
    fin
    maquinasPorZonaLibres[idZona] ← maquinasPorZonaLibres[idZona] − 1
    devolver idZonaYTiempos
fin
```

```
Función entrarEnBicicletaPremium()
    si bicicletaPremiumLibre = False entonces
        delay(colaBicicletaPremium)
    fin
    bicicletaPremiumLibre ← False
fin
Función salirDeBicicletaPremium()
    bicicletaPremiumLibre ← True
    resume(colaBicicletaPremium)
fin
Función salirDeZona(Y : Entero, idZona : Entero)
    maquinasPorZonaLibres[idZona] ← maquinasPorZonaLibres[idZona] + 1
    tiemposEnZonas[idZona] ← tiemposEnZonas[idZona] − Y
    resume(colasDeZonas [idZona])
fin
```

Algoritmo: Monitor Pantalla

```
Función usarPantalla(mensaje : String)
  // Imprimir mensaje en pantalla
fin
```

3.4. Cuestiones planteadas en los ejercicios

Según las condiciones de sincronización del problema,

a) **(0.5 puntos) Indica si la acción de entrenar es concurrente o en exclusión mutua y justifica.**
Es concurrente, ya que pueden haber varios procesos entrenando a la vez, siempre que hayan conseguido una máquina libre para ello. Como el entrenamiento se simula con un sleep, este segmento de código se puede realizar en paralelo junto con otros procesos que también estén entrenando, sin que esto suponga ningún problema.

b) **(0.75 puntos) ¿Qué tipo de monitor Java has usado? Justifica.**
He usado **condition**, ya que si uso synchronize no puedo usar más de una cola y no podría diferenciar entre procesos. Y como hay procesos que quieren acceder a diferentes zonas simuladas con colas es necesario usar condition. En el caso de la pantalla, como solo hay una y no necesito diferenciar entre procesos, podría usar synchronize, pero para mantener la consistencia en el código he decidido usar condition también para esta sección.

c) **(0.5 puntos) En el monitor, ¿has usado notify/signal o notifyAll/signalAll? Justifica**
Podría usar ambos, pero he decidido usar solo notify/signal, ya que no es necesario despertar a todos los procesos que esperan en una cola, porque al final solo uno de ellos va a poder ejecutarse y el resto se volverán a bloquear en la cola.

d) **(0.75 puntos) ¿Cómo se ha resuelto la exclusión mutua de la pantalla en este problema**
Está resuelto con el uso de un monitor que encapsula el uso de la pantalla en un procedimiento haciendo que solo un proceso pueda acceder a ella al mismo tiempo.

3.5. Código Java de los ejercicios comentado

Clase

4. Ejercicio 4: Paso de Mensajes Asíncrono

4.1. Recursos no compartibles

Los recursos no compartibles son las colas, tanto la rápida como la lenta y la pantalla.

4.2. Condiciones de sincronización

Las condiciones de sincronización se garantizan gracias al uso de un controlador que simula el paso de mensajes asíncrono, este tiene condiciones de guarda en los casos del uso de los recursos no compartibles. De esta forma, se garantiza la exclusión mutua mientras un proceso use uno de estos recursos.

En el caso de la asignación de un tiempo no es necesarío una condición de guarda porque no se esta protegiendo ningún recurso. Para liberar una cola y liberar la pantalla, no es necesario aplicar condiciones de guarda, ya que solo hará una de estas acciones un proceso que previamente halla accedido a uno de estos recursos.

Dicho todo esto solo es necesario aplicar condiciones de guarda para el uso de colas y el uso de pantalla.

Además al ser paso de mensajes asíncrono, en algunos casos, es necesario sincronizar al **controlador** con el **aficionado**, y al no haber **sends** bloqueantes se simulan con el uso de **receive** justo después del send, haciendo así que el proceso espere a la contestación del controlador. El uso de este mecanismo solo es necesario al pedir un tiempo, al pedir una cola y al pedir la pantalla. Para su liberación no es necesario, ya que el aficionado no necesita esperar a que el controlador le confirme que ha liberado el recurso.

4.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
Datos:
NUMERO_DE_AFICIONADOS : Entero
controladorDeAcceso : ControladorDeAcceso
aficionados : Aficionado[]
buzonesAficionados : Buzon[]
determinarColaConTiempo : Buzon
pedirColaRapida : Buzon
pedirColaLenta : Buzon
liberarColaRapida : Buzon
liberarColaLenta : Buzon
pedirPantalla : Buzon
liberarPantalla : Buzon
COLA_RAPIDA : Caracter
COLA_LENTA : Caracter
// Inicialización de variables globales
NUMERO_DE_AFICIONADOS ← 50
COLA_RAPIDA ← R
COLA_LENTA ← L
Función main()
    cobegin
        controladorDeAcceso.iniciar()
        para i ← 1 a NUMERO_DE_AFICIONADOS hacer
            // Inicialización de un hilo aficionado y
            // ejecución.
            aficionados[i].aficionado(i, buzonesAficionados [i]);
        fin
    coend
fin
```

```
Algoritmo: Clase Aficionado
Datos:
NUMERO_DE_ITERACIONES : Entero
idAficionado : Entero
caminarABanos : Entero
T : Entero
buzonAficionado : Buzon
NUMERO_DE_ITERACIONES ← 5
Función aficionado(idAficionado : Entero, buzonAficionado : Buzon)
    para i ← 1 a NUMERO_DE_ITERACIONES hacer
        sleep(random(1, caminarABanos))
        T ← random(1, 10)
        send(determinarColaConTiempo, {idAficionado, T})
        receive(BuzonAficionado, colaSeleccionada)
        seleccionar colaSeleccionada hacer
            caso COLA_RAPIDA hacer
                send(pedirColaRapida, idAficionado)
                receive(buzonAficionado, null)
                sleep(T)
                send(liberarColaRapida)
            fin
            caso COLA_LENTA hacer
                send(pedirColaLenta, idAficionado)
                receive(buzonAficionado, null)
                sleep(T)
                send(liberarColaLenta)
            fin
        fin
        send(pedirPantalla, idAficionado)
        receive(buzonAficionado, null)
        // Imprimir en pantalla
        send(liberarPantalla, null)
    fin
fin
```

```
Algoritmo: Clase ControladorDeAcceso
Datos:
libreColaRapida : Booleano
libreColaLenta : Booleano
librePantalla : Booleano

// Inicializar booleanos de guardas
libreColaRapida ← true
libreColaLenta ← true
librePantalla ← true

Función iniciar()
   while(true)
      select
           // Selector para pedir una cola.
           receive(determinarColaConTiempo, {idAficionado, T})
           si T ≤ 5 entonces
            send(buzonesAficionados [idAficionado], COLA_RAPIDA)
           en otro caso
            send(buzonesAficionados [idAficionado], COLA_LENTA)
         or  // Selector de cola rápida con guarda
           when libreColaRapida = true =>
           receive(pedirColaRapida, idAficionado)
           libreColaRapida ← false
           send(buzonesAficionados [idAficionado], null)
         or  // Selector de cola lenta con guarda
           when libreColaLenta = true =>
           receive(pedirColaLenta, idAficionado)
           libreColaLenta ← false
           send(buzonesAficionados [idAficionado], null)
         or  // Selector para pedir pantalla con guarda
           when librePantalla = true =>
           receive(pedirPantalla, idAficionado)
           librePantalla ← false
           send(buzonesAficionados [idAficionado], null)
         or   // Selector para salir de cola rápida
           receive(liberarColaRapida)
           libreColaRapida ← true
         or   // Selector para salir de cola lenta
           receive(liberarColaLenta)
           libreColaLenta ← true
         or   // Selector para salir de pantalla
           receive(liberarPantalla)
           librePantalla ← true
fin
```

4.4. Cuestiones planteadas en los ejercicios

a) **(0.5 puntos sobre 10) ¿Se pueden usar simultáneamente los dos tornos? Justifica la respuesta**
Si, ya que el controlador solo se encarga de dar acceso a uno de los dos si esta libre, por lo que puede darle acceso a dos procesos uno con el rápido y otro con el lento, siempre que ambos estén libres. Por lo que uno puede estar usando el lento hacer un **sleep** y mientras estar otro en el rápido haciendo lo mismo.

b) **(0.5 puntos sobre 10) ¿Cómo has resuelto la exclusión mutua de la pantalla?**
Con la asignación de esta por medio del conrolador y de una condición de guarda, evitando así que más de un proceso pueda acceder a ella a la vez.

4.5. Código Java de los ejercicios comentado

Clase